

Document Number: P1237R0

Date: 2018-10-08

Audience: SG16

Reply-to: Tom Honermann <tom@honermann.net>

SG16: Unicode meeting summaries 2018/07/11 - 2018/10/03

Summaries of SG16 meetings are maintained at <https://github.com/sg16-unicode/sg16-meetings>. This paper contains a snapshot of select meeting summaries from that repository.

- [July 11th, 2018](#)
- [July 25th, 2018](#)
- [August 29th, 2018](#)
- [October 3rd, 2018](#)

July 11th, 2018

Draft agenda:

- Discuss what we want to learn from Swift and WebKit developers.
- Potentially review papers from the Rapperswil post-meeting mailing.
- Review issues list and start identifying goals for San Diego.

Attendees:

- Artem Tokmakov
- Mark Zeren
- Tom Honermann
- Victor Zverovich

Meeting summary:

- Apologies to JeanHeyd Meneide and Steve Downey; It seems technical issues with BlueJeans prevented them (and others?) from joining the meeting. This issue and conflict with the World Cup semi-finals reduced attendance.
- Tom reconfirmed intent to rename our mailing list, but has not yet made progress on doing so.
- We then started reviewing some papers from the Rapperswil post-meeting mailing.
- [P0732R2: Class Types in Non-Type Template Parameters](#)
 - Tom asked if `std::text` and/or `std::text_view` should be literal types?
 - Tom noted this would require defining operator`<=>`.
 - Mark suggested adding a `std::text_literal`, but then asked about motivation:
 - `char8_t` allows differentiating encoding for standard mandated encodings. Is there a need to track encoding through non-type template parameters?
 - [P0784](#) would enable dynamic allocation for literal types, so a separate (non-allocating) type may not be required.
 - Victor asked why operator`<=>` is relevant.

- Tom explained that operator `<=>` is required for non-type template parameters, but defining it for text is problematic because it would be either expensive, or wrong for many use cases (e.g., because it would be code unit or code point based).
 - Tom suggested that `std::fixed_string` may suffice since `std::text_view` could be layered on top.
 - Mark observed a solution would still be needed for encoding tagging then.
- [P1030R1: std::filesystem::path view](#)
 - Tom mentioned that we had reviewed the earlier P0 revision during our May 30th meeting.
 - Tom noted that this revision addresses the concern we had with the char based interfaces requiring UTF-8 encoding. However, it addresses this by replacing the char based interfaces with `std::byte` based ones. This doesn't match existing practice for file name interfaces.
 - Tom mentioned that he would have liked to poll on this change, but since we didn't have a quorum, we would not do so. The poll would have been to restore the char based interfaces, but to match the encoding requirements for `std::filesystem::path`.
- [P1100R0: Efficient composition with DynamicBuffer](#)
 - Tom wondered if Mark wanted to look at this as potentially related to [P1010](#).
 - Mark responded that he felt it isn't strongly related.
- We then discussed Victor's recent [follow up email](#) regarding [P0645](#) and interpretation of field widths.
 - Mark stated that this is fundamentally a console problem, but that field widths are needed to implement programs like Eric Niebler's range based calendar example.
 - Mark also asked if we can specify that fill characters only consume one column of output.
 - Tom asked if we can rule out grapheme clusters as the unit of field width on the basis that the library must support non-Unicode encodings.
 - Victor suggested we could define an encoding agnostic concept of grapheme clusters. For Unicode, the concept is a 1x1 match with grapheme clusters. For other encodings, that concept might map to code points with no higher abstraction.
 - Tom replied that doing so is viable and that `text_view` would have to do so if its `Character` concept were to be redefined in terms of grapheme clusters.
 - Victor reiterated that he wants to implement both code point and grapheme cluster based approaches and explore use cases.
 - Tom observed that the concerns are effectively equivalent for consoles and text editors; assuming use of a monospaced font.
 - Tom asked if `format` is intended as a `printf` replacement.
 - Victor responded, yes, but that doesn't mean that we have to replicate prior mistakes.
 - Tom suggested an experiment: Take Eric's calendar program and modify it to display emojis for holidays; e.g., U+1F384 Christmas Tree on December 25th.
- Discussion then turned to questions we'd like to discuss with the Swift and WebKit teams.
 - JeanHeyd (absent due to technical problems), provided the following five questions via Slack:
 - JM1: How many bug reports are related to users incorrectly choosing which layer of abstraction to work with for Strings (code units / code points / grapheme clusters)?
 - Tom attempted a clarification; since Swift strings are grapheme cluster based, I think this question means, are users trying to do things at the grapheme cluster layer when they would be better served working at the code unit or code point level?
 - Mark posed the correlated question, how often do users try to work at code unit or code point level when they should just work at the grapheme cluster level?
 - JM2: Has the decision to use Extended Grapheme Clusters presented a problem (minor or major) in the usage?
 - Mark stated this should be the first question we ask.
 - Mark presented a different way of asking this question: What have been the best and worst results of this choice?
 - JM3: Has anyone ever wanted to pry underneath the string abstraction and perform their own set of text processing that wasn't supported by the language (e.g., retrieve code units / code points so they can do something that Swift did not let them do)? If so, does it happen often?

- Tom stated the answer to the first question is clearly yes. The second question is more about how often this happens and what the use cases are that motivate doing so.
 - *[Editor's note: a use case may be to work around differences in grapheme cluster boundaries in different Unicode versions depending on the version of Swift or the underlying version of ICU.]*
 - Mark expressed an interest in string builder use cases. How are custom string builders created?
 - JM4: Has Swift ever considered exposing lower-level unicode database code point / script properties? CharacterSet seems to have some of that functionality, but has more ever been requested / asked for?
 - Tom expressed enthusiasm for this question.
 - JM5: There's some indication that putting the normalization form and such in the type system may prove beneficial. Has there been any progress on that front? We are looking to answer a similar question for C++ up-front, and picking one normalization form that might have the most up-front processing and performance benefits for typical users.
 - Mark rephrased as, what was the rationale for choosing the current design?
- Tom then went over a list of questions he had come up with:
 - TH1: The Swift string manifesto is about 1 1/2 years old. What have you learned since?
 - TH2: If you were starting over, what would you change?
 - Tom stated that this isn't a very useful question; it's too open ended.
 - Mark stated that bug reports are more interesting; What have you had to change?
 - TH3: How tied is the Swift string implementation to ICU?
 - Tom stated the intent of this question is to identify how much of ICU is needed to create a useful Unicode string class.
 - Tom added a second goal: to determine if the Swift developers would potentially be interested in replacing uses of ICU with standard C++ library features, if they existed.
 - TH4: Swift's string is locale insensitive (yay!). Was a locale sensitive one considered? Perhaps as a distinct type?
 - Tom stated the intent is to explore if a distinct type for localized strings might be useful (since locale is a run-time property not available at compile-time).
 - TH5: How often does string interpolation suffice vs using string formatting?
 - Tom asked Victor if he had considered string interpolation support when designing his format library.
 - Victor responded, yes, but with uncertainty regarding how to do it in C++ today. Python started with a formatter and added interpolation later. We could do likewise.
 - TH6: Has canonical string equality been...
 - A performance issue?
 - A surprise to users?
 - TH7: Have substrings turned out to work as well as hoped?
 - Tom noted that Swift substrings seem superficially similar to `std::string_view`, but with dynamic lifetime management of the underlying storage.
 - TH8: Are the results of string interpolation always dynamic? Does Swift have a `constexpr` equivalent and, if so, do they work there?
 - TH9: Would you remove `string.count()` (returns "character" count) if you could?
 - Tom posed an additional question: How often do people use `string.count()` incorrectly?
 - TH10: Are the `unicodeScalars`, `utf8`, and `utf16` views allocating? Or are they lazy transformations?
 - TH11: There are a variety of "unsafe" methods. Have they been problematic?
- Mark suggested an additional question:
 - MZ1: Swift comparisons are provided. Do users use them incorrectly? Have they been a performance problem?
- Tom stated that our next meeting will be scheduled for July 25th.

July 25th, 2018

Draft agenda:

- Discuss the Unicode support experience with Swift and WebKit representatives (tentative pending their availability).
- Review our issues list and start identifying goals for San Diego.

Attendees:

- Artem Tokmakov
- JeanHeyd Meneide
- Mark Zeren
- Tom Honermann
- Zach Laine

Meeting summary:

- Tom announced that meeting with Swift developers was postponed due to scheduling conflicts and that, in the meantime, we'll focus on interaction with them over email. *[Editor's note: Michael Ilseman and Dave Abrahams responded to the initial set of questions. Their responses are available in the SG16 mailing list archive at <http://www.open-std.org/pipermail/unicode/2018-August/000113.html>].*
- Discussion then proceeded with review of the [SG16 issues list](#).
- [Issue #2: Deprecate std::ctype, std::ctype_byname, std::isupper\(\), and std::toupper\(\)](#).
 - Zach suggested writing a direction paper regarding deprecation policies.
 - Artem, observing that the indicated functions are used by iostreams (e.g., by std::uppercase), suggested we just go the extra mile and deprecate iostreams to a mixture of approval and laughter.
 - Mark suggested that the issue scope be limited to previously identified functions.
 - Tom agreed and renamed the issue (previously "Deprecate text/string/character interfaces that are too broken to fix").
 - Zach mentioned that isupper, isnum, and isalpha are definitely broken for Unicode and expressed a preference that, if we're going to deprecate them, we should do so early in order to encourage replacement.
 - Zach went on to explain that replacements that properly handle Unicode must take locale into account in order to do title casing and case mapping correctly.
 - Tom asked for clarification - a code point based toupper() doesn't make sense?
 - Zach responded, no; more information is needed.
 - Tom asked, what about isupper()?
 - Zach answered, Unicode properties can answer that question, but are insufficient for doing case conversions.
 - Tom summarized, the take away is that interfaces in <ctype> and <locale> are definitely broken.
 - Mark added, yup, especially considering that int is signed.
 - Artem asked about support for UTF-8, UTF-16, and UTF-32.
 - Mark replied, yup, those are problematic. Even for char32_t due to combining code points.
 - Tom stated this is not a high priority for C++20; no objections.
- [Issue #3: Uninitialized append for contiguous containers](#)
 - Mark noted that [P1010](#) was not presented in Rapperswil; hopefully it will be in San Diego.
- [Issue #4: basic_string specification cleanup](#)
 - Mark mentioned that Tim Song recently proposed some cleanup, but those changes don't address Mark's iterator invalidation concerns.
- [Issue #5: char8_t \(WG21 P0482, WG14 N2231\)](#)

- Tom stated that this is on target for C++20. Tom has some minor wording changes to make per request from early LWG review.
- Mark asked about the WG14 proposal.
- Tom replied that WG14 is meeting again in October and that he hopes to have a revision ready to present.
- [Issue #6: Specify that char16_t and char32_t literals are UTF-16 and UTF-32 respectively](#)
 - Tom indicated that the paper for this issue, [P1041R1](#), is ready for presentation in San Diego.
- [Issue #7: Modern terminology updates](#)
 - Zach observed that this is something that could be done for C++20 since the changes won't impact implementors.
 - Tom agreed but lamented a lack of time for working on it now.
- [Issue #8: Explicitly disallow unnamed Unicode codepoints in `http://eel.is/c++draft/lex.charset#2`](#)
 - Tom expressed a belief that this issue is complete. Martinho discussed it with CWG members in Rapperswil and submitted a [pull request](#) that was accepted as an editorial issue.
 - *[Editor's note: Tom was mistaken. The accepted pull request addressed a terminology issue ("short name" vs "short identifier"); the concern tracked by this issue remains, though Martinho has a draft paper [D1139](#) that addresses it.]*
- [Issue #9: Requiring `wchar\_t` to represent all members of the execution wide character set does not match existing practice](#)
 - Artem summarized: the standard requires that all members of the execution wide character set be representable in a single `wchar_t` value.
 - Zach stated a preference for treating this as low priority. Mark agreed.
 - Zach added that `wchar_t` is already a portability nightmare and there is therefore little incentive to try and fix it. Mark agreed.
- [Issue #15: Add support for named Unicode character escapes](#)
 - Tom indicated that the paper for this issue, [P1097R1](#), is ready for presentation in San Diego.
- [Issue #16: code point sequence\[view\]](#)
 - Tom mentioned that Lyberta, the individual that filed this issue, had also discussed it on the mailing list.
 - Zack asked for clarification regarding what this issue is about.
 - Mark summarized: this is the question of whether a text type should have `begin()` and `end()` members that iterate over grapheme clusters or code points or whether the type should not be a range, but provide explicit access to EGC and code point ranges.
 - Tom added that Lyberta had also wanted to expose differences between encoding schemes and encoding forms, though it seems this was driven by purity of design goals rather than use cases. Lyberta appeared to want to be able to, effectively, reinterpret cast a sequence of UTF16-BE code units (bytes) to a sequence of UTF-16 code units (`char16_t`). But that doesn't work (portably) because bytes and `char16_t` might be the same size.
 - Mark commented, well that is fine, but don't put that in the standard then. That's why we like C++; it lets you break the rules.
- [Issue #30: Unclear behavior for octal and hex escape sequences in Unicode character and string literals](#)
 - Tom expressed a preference for making character literals like `u8'\x80'` well-formed; this matches existing practice.
 - Zach disagreed and presented the perspective that `u8`, `u`, and `U` literals should always produce well-formed UTF sequences.
 - Tom objected with the observation that `u8'\x80'` can't produce well-formed UTF-8 since it only produces a single code unit.
 - Zach suggested that perhaps `u8'\x80'` should be allowed, but `u8"\x80"` should not be.
 - Mark stated that both should be allowed because the programmer explicitly used a hex (or octal) escape sequence.
 - Zach objected saying that if he were to use an escape sequence that he wants the compiler to validate it.
 - Mark admitted seeing Zach's point.

- Zach stated that, if a programmer wants to create an ill-formed sequence for some reason, then they should use `bit_cast` from a `char` sequence after creating the data. The intent of adding a `u8` prefix to a string is to request well-formed UTF-8.
- Tom disagreed and stated the intent of adding a `u8` prefix is to enable transcoding from the source character set to UTF-8.
- Mark noted that this distinction is important due to planned changes for `char8_t`.
- Tom disagreed and stated this is orthogonal since it is independent of the type system.
- Tom noted that we can address this as a core issue or by writing a paper.
- Mark said we should write a paper since there are different options for what the behavior should be. Zach agreed.
- Tom suggested that a core issue be filed to address the difference in what the standard states and in what current implementations actually do. A separate paper can then address what the desired behavior is.
- Zach stated that he doesn't think a defect report suffices to address this.
- Tom stated that he'll file a core issue; Zach and Mark can follow up with a paper.
- Mark mentioned that Martinho has a stake in this; that he wanted hex and octal escapes to be a back door.
- JeanHeyd confirmed and agreed that hex and octal escapes should function as back doors. If a programmer wants to ensure well-formed UTF, use `\u` or `\U` or (hopefully soon), `\N{}`.
- [Issue #31: std::text and std::text_view](#)
 - Tom: On-going.
- [Issue #32: std::char_traits<char16_t>::eof\(\) requires uint_least16_t to be larger than 16 bits \(LWG#2959\)](#)
 - Tom summarized: All 16-bit values are valid UTF-16 code units. This doesn't leave any room for a 16-bit value to be used to indicate EOF. Implementations often use `0xFFFF` to indicate EOF. The result is spurious mismatches with `std::char_traits<char16_t>::eof()` when text encodes (valid) UTF-16 `0xFFFF` code units.
 - Zach observed that this isn't solvable without switching to a larger `int_type`.
 - Tom agreed but noted that it is an ABI break.
 - Tom added that `libstdc++` made a change to minimize problems by mapping `0xFFFF` code units to `0xFFFFD` when comparing against `eof()`, but this doesn't solve the problem.
- Tom asked what should be on the list for C++20. Our `char8_t`, `char16_t` and `char32_t` literals are UTF-16/UTF-32, named escape sequences, and uninitialized string append proposals are underway. We could make progress on other issues or work towards C++23 goals like `std::text` and `std::text_view`.
- Zach observed that the direction group would likely prioritize feature work over existing issues.
- Tom agreed and summarized, it sounds like prioritize features, resolve issues opportunistically.
- Zach then provided an update on `Boost.Text`. He expects to have it ready for submission for Boost review soon; David Sankel has agreed to assist.
- Zach added that he got collation based text searching working and that it was fun because he could use Boyer-Moore searching for it. He asked if any of us had used full collation based searching before.
- Artem responded that most people want linguistic searching; for example, searches for "frog" return "toad".
- Mark observed that linguistic searching goes a bit beyond Unicode.
- JeanHeyd asked if we should be considering exposing the Unicode character database. Python and Java do *[Editor's note: and the next version of Swift will]*.
- Tom was unsure and noted that programmers need for properties like "is number" and "is space" often have more strict constraints than Unicode; e.g., when parsing some mini-language.
- Zach added that, for full text processing, you're generally not looking at those properties either.
- Mark observed that adding the timezone database nearly made some committee members oppose the feature due to the extra 1MB or so of size.

August 29th, 2018

Draft agenda:

- SG16 direction. Where are we heading? Big picture.
- Code points, EGCs, or explicit ranges for text views/containers?
 - How to decide? Pick a direction now? Write a pros/cons paper for the committee?

Attendees:

- Artem Tokmakov
- JF Bastien
- Mark Zeren
- Peter Bindels
- Steve Downey
- Tom Honermann
- Zach Laine

Meeting summary:

- With apologies from the editor, this summary writeup was very much delayed.
- Zach started off with an update on Boost.Text. He noted that implementing the Unicode bidirectional algorithm was challenging. Noone was surprised.
- Tom provided a brief summary for the agenda. Basically to review our direction and confirm common goals and scope.
- JF asked what we have planned for C++20 to which Tom replied that we have a few small features in the queue and might otherwise take on some wording cleanup.
- Steve asked about timing for a potential TS and discussion ensued regarding how to get usage experience vs the benefits of going straight into the standard.
- Tom proposed a few statements to be considered as axioms, guidelines, questions, or possible directives for our work.
- (Axiom) 1: C++ has a long history of supporting non-Unicode encodings; we can't abandon legacy encodings.
 - JF brought up the concept of bridging with a comparison to `std::thread` and `native_handle`. E.g., an interface could provide a Unicode centric interface that abstracts support for legacy encodings.
- (Axiom) 2: execution and wide execution character encoding will remain run-time properties, `char8_t`, `char16_t`, and `char32_t` encodings will remain compile-time properties.
 - Tom asserted that legacy compatibility prevents mandating that the execution and wide execution encodings be fully known at compile time and noted that they can be changed dynamically by calling `set locale`.
 - Tom also noted that WG14 is considering allowing a program's locale to be dynamically changed on a per-thread basis. See [WG14 N2226](#).
 - Artem asked how much we've been looking at existing locale support.
 - Zach responded that the existing locale support is insufficient to implement some parts of Unicode, in particular, support for tailoring.
 - JF mentioned that Javascript internationalization may be a good resource with regard to how to map locale information to Unicode.
- (Guideline) 3: Encourage the internal vs external encoding model with UTF-8 as the preferred internal encoding.
 - Tom asked if it is reasonable to encourage use of a particular encoding as the internal encoding.
 - Zach replied that he feels we must in order to avoid having to perform internal conversion rather than (only) conversions at component boundaries.
 - Mark suggested that extensions could enable support for other encodings.
 - Peter emphasized existing advocacy and trends with regard to UTF-8:

- <https://utf8everywhere.org>
- https://w3techs.com/technologies/overview/character_encoding/all
- Tom asked JF if he could comment regarding how UTF-8 fits into the Apple ecosystem.
- JF responded that, as long as convenient transcoding interfaces are available, that it wouldn't be an issue.
- Tom asked if restricting access to code units in `std::text` (in order to allow the internal encoding to be implementation detail) would break use cases.
- Zach responded yes, that prevents passing the underlying code unit sequence to C APIs. *[Editor's note: this response presumes that the underlying code unit sequence contains a null terminator]*
- (Directive) 4: Improve support for transcoding at program borders (command line, env vars, stdin, stdout, text files, network).
 - Zach suggested not focusing on improving this now; let `fmt` deal with I/O; don't enhance `iostreams`.
 - Mark stated that we don't have to fix all of the problems with the standard library.
- (Question) 5: Do `std::text` and `std::text_view` replace `std::string` in new programs?
 - Mark stated no, not as a drop in replacement.
 - Zach noted that we want to continue using `std::string` for simple cases.
 - Tom asked, for new code, do we advocate a preference for `std::text` and `std::string` only when needed?
 - Zach stated no, for performance reasons.
 - Tom clarified: that indicates a specific reason to prefer `std::string` in some context, but in general, can we advocate use `std::text` unless there is a reason not to?
 - Zach responded that an AAT (Almost Always Text) rule would make sense.
 - Peter asked if it would ever be wrong to use `std::text` instead of `std::string`.
 - Zach replied, no.
 - Peter provided an example by way of `set<text>`. If `std::text` comparisons are expensive (e.g., canonical equivalence vs lexicographical), use as a container element may not be desirable.
 - Zach noted that might be a reason to specialize `std::less`.
 - Zach observed that comparison cost is only an issue for relational comparison, equivalence is inexpensive if the text is already normalized.
 - Mark summarized, `std::text` provides storage, comparisons need specialized support.
- (Question) 6: How do we manage `std::text` and `std::string` conversions?
 - Tom asked if we need the ability to transfer buffer ownership between `std::string` and `std::text`
 - Mark replied, yes, and that it needs to handle short buffer optimizations, but that this is lower priority than making the Unicode algorithms available.
 - Artem observed that `std::string_view` helps here.
- (Question) 7: Where do null terminated strings fit in?
 - Tom asked, can we try to reduce demand for them? Perhaps propose a `string/text` type to WG14?
 - Everyone replied, not quickly :)
 - Mark asked if `std::text` needs null termination.
 - Zach replied that it can be provided at the code unit level for C compatibility, but doesn't make sense to provide null termination for code point or grapheme cluster sequences.
- (Question) 8: Where do Unicode algorithms fit into the library and are they independent of `std::text`?
 - Tom stated a preference that Unicode algorithms are usable with arbitrary string types.
 - Zach agreed stating that we should have code point range/iterator based interfaces as well as grapheme cluster range based interfaces.
- (Directive) 9: Adopt useful features from other languages.
 - Tom clarified, for example, named escapes as proposed in [P1097](#).
 - No disagreement.
- (Directive) 10: Fix existing issues as needed.
 - No disagreement.
- (Question) 11: What role do we take with WG14?
 - Tom asked, the question is really how much time to spend here.
 - Zach stated that engaging with WG14 over `char8_t` and terminology updates makes sense.
 - Mark observed that making Unicode data available via a C API could be useful.

- (Question) 12: What is our target schedule?
 - Steve suggested mostly targeting C++23, not a TS.
 - Zach noted that we need to ensure usage experience and that we have bandwidth limitations.

October 3rd, 2018

Draft agenda:

- Last meeting before the San Diego pre-meeting mailing deadline on October 8th.
- Review the draft SG16 direction paper that Tom plans to have ready for this meeting and the pre-meeting mailing.
- Code points, EGCs, or explicit ranges for text views/containers?
 - How to decide? Pick a direction now? Write a pros/cons paper for the committee?

Attendees:

- Artem Tokmakov
- Corentin Jabot
- JeanHeyd Meneide
- Mark Zeren
- Markus Scherer
- Steve Downey
- Tom Honermann
- Zach Laine

Meeting summary:

- We started off with a round of introductions in honor of a new first time attendee, Markus Scherer, chair of the ICU Technical Committee.
- Tom provided a brief overview of the agenda; to review draft papers discussing SG16 direction, to collect feedback, and submit a paper for the San Diego pre-meeting mailing that represents the group's consensus on our general direction.
[Editor's note: these drafts later became [P1238R0](#)]
- Zach raised a concern regarding support for generic interfaces. The draft paper asked whether generic interfaces for Unicode algorithms could reasonably support segmented data structures like ropes. Zack felt segmented data structures are supported naturally as long as they provide standard iterators.
- Tom explained that the question was meant more to ask if generic interfaces could provide performance that users would expect. Or whether interfaces specialized for contiguous memory would be necessary and, if so, whether they could be used to service ropes. Perhaps it would make sense to have a low level C API wrapped in a generic interface. This would require the low level API to support tracking state (e.g., code unit sequences split across segment boundaries).
- Zach expressed concern about giving the impression that we want to provide equivalent functionality in C and C++.
- Corentin chimed in that contributing to C isn't something we've talked much about.
- Tom clarified, only when it makes sense.
- Markus noted some experience; prior attempts to provide generic interfaces in ICU resulted in performance complaints. ICU could do more of this, but users are able to do it themselves.
- Zach responded that his own performance tests involving arrays of code points vs code point iterators on top of code units indicated negligible performance differences. Table lookups dominated.
- Markus commented that performance improvements come about largely due to support for fast paths.
- Mark observed that we heard similarly from Swift developers regarding the need to support fast paths.

- Markus then asked a fundamental question: why bother standardizing Unicode support? Why not just use ICU?
- Mark responded that programmers continue to struggle with classes of bugs that we could potentially minimize, handling of grapheme clusters for example.
- Steve also noted continued mishandling of strings in general.
- Tom mentioned distribution and packaging issues. Having something provided with the standard library helps to sidestep legal obstacles and package versioning problems.
- Corentin commented that programmers need more easy to use functionality, libraries that encourage correct use.
- Tom agreed, noting that we want to bring down the learning curve for working with Unicode.
- JeanHeyd added that not all programmers need all of Unicode, some would benefit just by having support for encodings built in.
- Changing topics, Mark asked to add a reference to P1072 in the paper, noting its relevance to text/string buffer transference.
- Steve asked about some of the terminology in the paper. Why the inconsistent mention of UTF-8 vs `char16_t` and `char32_t`?
- Tom explained that this is consistent with the standard where `u8` literals are explicitly UTF-8, but `u`, `U`, and other uses of `char16_t` and `char32_t` currently have implementation defined encodings.
- Corentin observed that `char16_t` and `char32_t` are explicitly used for UTF-16 and UTF-32 respectively in the filesystem library.
- Changing subjects again, Tom asked for thoughts regarding the first constraint in the paper, that the ordinary and wide execution encodings are implementation defined. Can we lift that constraint?
- Tom went on, Microsoft is working on adding better UTF-8 support to Windows and their compiler. IBM does not provide a publicly available C++11 compliant compiler for z/OS, though they do provide Swift on z/OS and that depends on Clang. IBM doesn't publicly provide Clang on z/OS, but it seems they have an internal port of it.
- Markus noted that ICU dropped support for IBM's z/OS, i, and AIX operating systems when upgrading to C++11 due to lack of C++11 support in IBM's xLC compiler.
- Corentin mentioned that we're targeting C++23 or C++26 for our work. What will things look like then?
- Changing topics again, Markus commented on ICU's switch to using `char16_t` as the code unit type for its internal encoding. This was challenging due to interoperability issues with code that used, and continues to use, `wchar_t` or `uint16_t` for UTF-16 data. Overloads were added to make it easier to integrate with code using these types.
- Tom asked to confirm his historical understanding, that ICU used to use a typedef for the code unit type that consumers could set to `wchar_t` or `uint16_t` as required for their application.
- Markus confirmed that users can still do so, but that the default is now `char16_t` when compiling as C++11.
- Zach asked to talk about UTF-8 and type safety. He was recently surprised when, due to a mismatch between the encoding used for a source file (UTF-8) and the encoding the compiler used to read that source file (Windows 1252), `u8` string literals didn't have the expected contents at run-time. He concluded (accurately) that he can't depend on `u8` string literals containing well-formed UTF-8 text. This caused him to question his perception of the type safety that `char8_t` provides.
- Markus expressed further concerns about `char8_t` leading to the same type interoperability issues that were encountered with `char16_t` in ICU.
- Mark noted that we are still lacking deployment results with `char8_t`.
- JeanHeyd described prior experience using a `char8_t` like type to help avoid encoding confusion and that it was useful.
- Tom stated that he will add discussion of `char8_t` to the agenda for the next meeting and update discussion in the direction paper.
- Changing topics, Markus mentioned a wish list item, that `char` be made unsigned everywhere.
- Mark thought floating the idea would be worthwhile.
- Tom asked Steve about merging the two draft papers. Steve was favorable to the idea.
- Steve also mentioned that the paper needs to discuss concerns with allocators. Tom agreed.
- Mark expressed a desire to discuss allocators in San Diego.

- Steve also suggested that the paper address the expected delivery time for features we're discussing. In particular, to make it clear that `std::text` is not targetting C++20.
- Tom agreed. Mark stated the paper should also address the intended target for existing papers in flight.